FEEDBRIDGE BOT

Intelligent RSS-to-Chat Delivery Platform

Product Requirements Document
Version 1.0

Document Type	Product Requirements Document (PRD)
Status	Draft for Review
Date	July 17, 2026
Primary Stack	Go (preferred) or Rust
Secondary / Frontend	JavaScript / TypeScript (Next.js)
Target Platforms	Telegram (MVP) → Discord → WhatsApp

This document defines the product vision, features, technical architecture, and implementation roadmap for FeedBridge — a self-hostable, AI-enhanced feed delivery bot that turns any website URL into personalized chat notifications.

Prepared for: Project Founder & Future Contributors
Stack Philosophy: Performance-critical core in Go/Rust; user-facing surfaces open to JS for easy onboarding.

1. Table of Contents

2. Product Vision & Overview
3. Goals & Success Metrics
4. Target Users & Personas
5. User Stories & Use Cases
6. Functional Requirements (Features)
7. Non-Functional Requirements
8. Technical Architecture
9. Technology Stack (Go/Rust + JS parts)
10. Data Model
11. AI Integration Layer
12. Feed Detection & Processing Pipeline
13. Platform Integrations
14. Development Roadmap & Phases
15. Risks, Mitigations & Open Questions
16. Recommendations
17. Appendix

2. Product Vision & Overview

2.1 Vision Statement

FeedBridge empowers users to reclaim control over their information diet. By accepting any website URL, intelligently detecting or generating feeds, applying AI-powered filtering and summarization, and delivering clean updates into the chat apps people already use, FeedBridge eliminates algorithm noise and notification fatigue.

2.2 Problem Statement

• Social media and news apps bury valuable content under engagement-maximizing algorithms.
• Most people do not know how to find or use RSS/Atom feeds.
• Existing RSS readers are often desktop-only or require another app/install.
• Chat is where attention already lives (Telegram, Discord, WhatsApp).
• Users want filtering and summarization without learning complex tools.

2.3 Solution Summary

A conversational bot that:
1. Accepts a URL from the user.
2. Detects existing RSS/Atom feeds or offers generation fallbacks.
3. Guides subscription with simple prompts (frequency, filters, AI preferences).
4. Polls feeds efficiently and delivers new items as rich chat messages.
5. Uses AI for relevance scoring, keyword/topic filtering, and summarization.
6. Provides natural-language management commands.

2.4 Key Differentiators

Differentiation	Description
URL-first UX	No need for users to know RSS URLs — just paste any page.
AI-native filtering	Relevance, summarization, and topic control via conversation.
Chat-native delivery	Lives where users already spend time.
Self-hostable core	Privacy-first; single binary or Docker image.
Polyglot-friendly	Core in Go/Rust; web dashboard & marketing open to JS.
YouTube & special sources	First-class support for common hard-to-find feeds.

3. Goals & Success Metrics

3.1 Product Goals

• Deliver a reliable MVP Telegram bot within 4–6 weeks of focused development.
• Achieve high feed-detection success rate (>85% of popular news/blog sites).
• Provide meaningful AI value (users report reduced noise / better relevance).
• Keep the core service lightweight enough for cheap self-hosting (single VPS or even home server).
• Make the codebase approachable for future collaborators (clear modules + JS surfaces).

3.2 Success Metrics (KPIs)

Metric	MVP Target	6-Month Target
Active subscriptions	100+	5,000+
Feed detection success rate	≥ 80%	≥ 92%
Avg. items delivered / user / day	3–8	5–12 (filtered)
Unsubscribe rate (7-day)	< 25%	< 15%
AI filter usage rate	≥ 40% of users	≥ 65%
Self-host deployments	10+	200+

4. Target Users & Personas

Alex — The Power User

Developer / researcher who follows 20+ blogs, YouTube channels, and GitHub releases. Wants zero noise and full control. Comfortable with bots and self-hosting.

Sam — The Casual Consumer

Follows a few news sites and creators. Does not know what RSS is. Wants simple “paste link → get updates in Telegram” experience with optional AI summaries.

Jordan — The Community Admin

Runs a Discord server or Telegram channel and wants to automatically share relevant updates from external sources with members (with moderation filters).

Priya — The Privacy Advocate

Avoids big tech algorithms. Prefers self-hosted tools. Will run FeedBridge on a home server or cheap VPS and share the instance with trusted friends.

5. User Stories & Use Cases

5.1 Core User Stories (MVP)

- As a user, I can send any website URL to the bot so that it attempts to find a feed for me.
- As a user, I can see a preview of the detected feed (title, sample items) before committing.
- As a user, I can choose polling frequency (15 min / 1 h / 6 h / daily) when subscribing.
- As a user, I receive new feed items as clean chat messages containing title, short summary, and link.
- As a user, I can list my active subscriptions and unsubscribe from any of them with a simple command.
- As a user, I can mute a feed temporarily or permanently.

5.2 Intelligence Stories (Phase 2)

- As a user, I can tell the bot “only send me AI and tooling articles” and it will filter accordingly.
- As a user, I can request a TL;DR summary instead of (or in addition to) the original abstract.
- As a user, I can ask the bot “why did you send this?” and get a short relevance explanation.
- As a user, I can set a daily digest mode instead of real-time notifications.

5.3 Admin / Operator Stories

- As an operator, I can view aggregate metrics (feeds polled, items delivered, error rates).
- As an operator, I can black-list abusive domains or rate-limit individual users.
- As a self-hoster, I can configure everything via environment variables and a single config file.

6. Functional Requirements (Features)

6.1 Must-Have (MVP)

ID	Feature	Description
F-01	URL Ingestion	Accept plain URLs, YouTube channel/playlist links, Reddit, etc.
F-02	Feed Auto-Discovery	Parse HTML <link rel="alternate"> + common paths (/feed, /rss.xml, /atom.xml, etc.).
F-03	Special Source Handlers	YouTube (channel_id → videos.xml), basic Reddit support.
F-04	Subscription Flow	Preview → confirm → set frequency → store.
F-05	Polling Engine	Configurable cron-like polling with ETag / Last-Modified support.
F-06	Item Deduplication	Track guids / link hashes per subscription to avoid repeats.
F-07	Message Formatting	Rich Telegram messages (Markdown / HTML) with title, snippet, link.
F-08	Basic Commands	/start, /help, /list, /unsub, /mute, /status.
F-09	Error Feedback	Clear messages when no feed found or feed is broken.
F-10	Persistence	Users, subscriptions, last-seen items, and settings stored reliably.

6.2 Should-Have (Phase 2)

- AI relevance scoring & keyword/topic filters
- AI-generated summaries / TL;DR
- Natural language filter commands
- Daily / weekly digest mode
- Full-article extraction (readability) when feed has only a snippet
- Basic web dashboard (subscription list, stats)
- Discord bot support
- Export / import of subscriptions (OPML-like)

6.3 Could-Have (Later)

- WhatsApp support (experimental / unofficial)
- Multi-user shared feeds (one poll → many subscribers)
- Custom notification templates
- Webhook / API for advanced users
- Feed generation via third-party services (rss.app, etc.) as fallback
- Collaborative filtering suggestions (“users like you also follow…”)
- Mobile-friendly progressive web app dashboard

7. Non-Functional Requirements

Category	Requirement
Performance	Poll 1,000 feeds in under 60 s on a modest 2-vCPU VPS. Message latency < 5 s after item detection.
Reliability	99% uptime target for the bot service. Automatic retry with exponential backoff on feed failures.
Scalability	Horizontal scaling via worker processes / queues. Shared feed pool to avoid duplicate polling.
Security	Secrets via env vars. Rate limiting on user commands. No execution of feed content. Input sanitization.
Privacy	Minimal data collection. Optional self-hosting. Clear data retention policy. No selling of feed data.
Observability	Structured logging (JSON). Prometheus metrics endpoint. Health check route.
Deployability	Single Docker image + docker-compose. One binary for Go/Rust builds. Clear env documentation.
Maintainability	Modular packages. Clear separation of bot interface, feed engine, AI layer, and storage.
Cost	MVP can run on free/cheap tiers. AI costs kept optional and user-controllable.

8. Technical Architecture

8.1 High-Level Components

The system is deliberately modular so that the performance-critical pieces stay in Go or Rust while presentation and optional dashboards can be written in JavaScript/TypeScript.

Component	Language Preference	Responsibility
Bot Interface Layer	Go / Rust	Telegram / Discord adapters, command parsing, message sending
Feed Engine	Go / Rust	Discovery, polling, parsing, deduplication, scheduling
AI Service	Go / Rust (HTTP client)	Calls to LLM APIs for scoring, filtering, summarization
Storage Layer	Go / Rust + SQL	Users, subscriptions, items, settings (Postgres / SQLite)
Worker / Queue	Go / Rust	Background polling jobs, retry logic
Web Dashboard (optional)	JavaScript (Next.js)	User-facing management UI, analytics, onboarding
Marketing / Landing	JavaScript	Public site, docs, status page
Admin / Metrics	Go/Rust + optional JS	Prometheus, health, simple admin endpoints

8.2 Data Flow (Happy Path)

1. User sends URL → Bot Interface validates and hands off to Feed Engine.
2. Feed Engine performs discovery (HTML + heuristics + special handlers) → returns feed URL(s).
3. User confirms → Subscription record created in Storage.
4. Scheduler / Worker periodically loads due subscriptions → polls feed (with ETag).
5. New items are deduplicated → optionally passed through AI Service for filtering/summary.
6. Formatted messages are sent back via Bot Interface to the user’s chat.

8.3 Deployment Topology

• Single-binary or Docker container for the core service (bot + feed engine + workers).
• Optional separate Next.js frontend talking to a small REST/gRPC API exposed by the core.
• Postgres (or SQLite for tiny installs) as the system of record.
• Optional Redis for distributed locking / job queues when scaling beyond one instance.

9. Technology Stack

9.1 Core (Go Preferred / Rust Alternative)

Area	Go Recommendation	Rust Recommendation
Language	Go 1.22+	Rust 1.78+ (edition 2021)
HTTP / Web	Fiber or Gin	Axum or Actix-Web
Telegram Bot	go-telegram-bot-api / telebot	teloxide
Discord	discordgo	serenity / twilight
Feed Parsing	mmcdole/gofeed	feed-rs
HTML Parsing	goquery / net/html	scraper / html5ever
Database	sqlx or pgx / bun	sqlx or diesel / sea-orm
Migrations	golang-migrate / goose	sqlx migrate / refinery
Scheduler	robfig/cron + own workers	tokio + cron crates
Config	envconfig / viper	config / figment / envy
Logging	slog / zerolog	tracing + tracing-subscriber
Metrics	prometheus client	prometheus crate
Testing	testing + testify	cargo test + mockall

9.2 JavaScript / TypeScript Surfaces (for future onboarding)

These parts are intentionally left open so that contributors who prefer the JS ecosystem can still contribute meaningfully without touching the Go/Rust core:

- Web Dashboard / User Settings UI (Next.js + React + Tailwind)
- Public Landing Page & Documentation site (Next.js or Astro)
- Optional Admin UI for operators
- Marketing analytics / status page
- Future: lightweight Node micro-service for non-critical tasks (if ever needed)
- Any browser extensions or companion tools

Recommendation: Keep a thin, well-documented REST (or gRPC-web) API on the core service. This allows the Next.js dashboard to be developed completely independently and makes onboarding JS developers straightforward.

10. Data Model

10.1 Core Entities (simplified)

User
id, platform (telegram/discord/…), platform_user_id, username, created_at, settings (JSON), is_premium, last_active

Feed
id, canonical_url, title, site_url, feed_type (rss/atom), etag, last_modified, last_checked, last_success, error_count, metadata (JSON)

Subscription
id, user_id, feed_id, frequency_minutes, filters (JSON — keywords, topics, min_score), ai_mode (off/filter/summarize), is_muted, digest_mode, created_at, last_notified

Item (or FeedItemSeen)
id, feed_id, guid/hash, title, link, published_at, first_seen_at — used for deduplication across all subscribers of that feed.

DeliveryLog (optional)
id, subscription_id, item_id, delivered_at, status, ai_score, summary_used — for analytics and debugging.

10.2 Design Notes

- Normalize feeds so multiple users sharing the same source only cause one poll.
- Store filter preferences and AI settings as flexible JSON for rapid iteration.
- Use UUIDs or snowflake IDs for distributed friendliness.
- Keep a soft-delete / archive strategy for old items to control storage growth.

11. AI Integration Layer

11.1 Capabilities

• **Relevance Scoring** — given user preferences + item title/summary, return 0–1 score.
• **Keyword / Topic Filtering** — extract topics or match against user-defined lists.
• **Summarization** — produce a short (1–3 sentence) TL;DR suitable for chat.
• **Natural Language Understanding** — interpret commands such as “mute politics from CNN”.

11.2 Design Principles

• AI is optional and user-controlled (cost + privacy).
• Prefer small, cheap models (Groq, Gemini Flash, Claude Haiku, GPT-4o-mini) for volume scoring.
• Cache results aggressively (same item + same preferences → reuse score/summary).
• Fail open: if AI is down, fall back to raw feed items rather than blocking delivery.
• Log enough context for later evaluation of filter quality without storing full content forever.

11.3 Prompt & Cost Control

Keep prompts short and structured. Use a two-stage approach when volume is high: cheap model for filtering → more capable model only for items that pass the filter and need a polished summary. Expose a simple “AI budget” or “max AI calls per day” setting per user.

12. Feed Detection & Processing Pipeline

12.1 Detection Strategy (ordered)

1. Special-case handlers (YouTube channel ID extraction, Reddit, GitHub releases, etc.).
2. Look for <link rel="alternate" type="application/rss+xml|atom+xml"> in HTML head.
3. Probe common paths: /feed, /rss, /rss.xml, /atom.xml, /feeds/posts/default, /index.xml, etc.
4. Heuristic parsing of page for feed-like links.
5. Optional: call external feed-finder services or generation APIs as last resort (user-confirmed).

12.2 Polling Best Practices

- Always send If-None-Match / If-Modified-Since when previous ETag or Last-Modified is known.
- Respect Retry-After and Cache-Control when present.
- Exponential backoff + circuit breaker for persistently failing feeds.
- Jitter polling times so many feeds do not fire at the same second.
- Cap concurrent fetches (connection pool + semaphore).

12.3 Content Handling

- Prefer full content when the feed provides it; otherwise store only the summary + link.
- Optional later stage: fetch the article page and run a readability algorithm for better summaries.
- Strip or sanitize HTML aggressively before sending to chat platforms.

13. Platform Integrations

13.1 Telegram (Primary for MVP)

• Long-polling or webhook mode (webhook preferred for production).
• Rich messages with MarkdownV2 or HTML, inline buttons for quick actions (Unsubscribe, Mute, Digest).
• Excellent bot UX and wide user base. Easiest path to real usage.

13.2 Discord

• Slash commands + interactions for modern UX.
• Can post into user DMs or into server channels (with permission checks).
• Good for community use-cases (Jordan persona).

13.3 WhatsApp (Later / Experimental)

• No official free bot API for normal numbers. Options: WhatsApp Business Cloud API (requires approval + payment) or unofficial libraries (risk of bans). Treat as experimental and clearly label it as such.

13.5 Telegram Delivery Modes (Recommended Hybrid)

13.5.1 Overview

The recommended approach is a **hybrid model** that supports both simple and advanced users while leveraging Telegram's native Forum Topics feature:

Mode	Description	Best For
Private Chat (Default)	All commands and feed updates happen in the 1:1 chat with the bot.	Casual users (MVP)
Forum Topics (Advanced)	Bot creates a dedicated Topic per feed inside one Forum Supergroup.	Power users & communities

13.5.2 User Flow

1. User starts in private chat with the bot (control center).
2. When adding a feed, bot offers two options:
• Send here (private chat — simple mode)
• Create Topic in my Forum group (hybrid mode)
3. For hybrid mode, user forwards a message from their Forum group so bot can get chat_id.
4. Bot calls createForumTopic() and stores mapping (subscription → topic_id).
5. Future items are posted using message_thread_id to the correct topic.

13.5.3 Technical Notes

• Requires bot to have can_manage_topics permission in the group.
• Store delivery_chat_id and message_thread_id per subscription.
• Falls back gracefully if topic creation fails.

13.5.4 Why This Hybrid?

Combines simplicity for casual users with powerful organization for power users. Uses native Telegram features (no need for multiple separate chats).

14. Development Roadmap & Phases

Phase 0 — Foundation (Week 1)

• Project scaffolding (Go or Rust), config, logging, DB connection, migrations.
• Basic Telegram bot that replies to /start and /help.
• Simple feed parser test harness.

Phase 1 — MVP Core (Weeks 2–4)

• Full feed discovery + subscription flow.
• Polling engine with ETag support and deduplication.
• Message delivery + core commands (/list, /unsub, /mute).
• Docker packaging and basic documentation.
• Soft launch to a small group of friends / early testers.

Phase 2 — Intelligence (Weeks 5–8)

• AI relevance scoring and filter preferences.
• Summarization option.
• Natural language filter commands.
• Digest mode.
• Improved YouTube and other special handlers.

Phase 3 — Polish & Multi-Platform (Months 3–4)

• Discord support.
• Basic web dashboard (JS).
• Metrics, better error UX, OPML export/import.
• Self-hosting guide and public release.

Phase 4 — Scale & Advanced (Later)

• Shared feed pool + advanced queuing.
• WhatsApp experimental path.
• Premium / paid AI features if desired.
• Community contributions and plugin system ideas.

15. Risks, Mitigations & Open Questions

Risk	Impact	Mitigation
Feed sites block or rate-limit the bot	High	Respectful User-Agent, ETags, low concurrency, per-domain backoff, optional proxy support later.
AI costs spiral	Medium	Make AI optional, cache heavily, use cheap models, expose budget limits to users.
WhatsApp ban risk	High for that platform	Keep unofficial support experimental; prefer Business API when ready.
Low feed detection success	High	Special handlers for top 50 sites, continuous improvement of discovery, clear “no feed found” UX with alternatives.
Spam / abuse of the bot	Medium	Rate limits, simple CAPTCHA-like flows if needed, admin blacklists.
Scope creep	Medium	Strict MVP definition; park nice-to-haves in the backlog.
Contributor friction (mixed languages)	Low–Medium	Clean API boundaries + excellent docs + JS dashboard as the easy onboarding path.

15.1 Open Questions

- Will the project be fully open-source from day one, or core open + hosted premium later?
- Preferred LLM provider for the default AI path (cost vs quality vs privacy)?
- How aggressively should full-article scraping be pursued (legal / ToS considerations)?
- Multi-tenant hosted SaaS version in the future, or pure self-host focus?

16. Recommendations

The following recommendations are explicitly tagged so they can be accepted, modified, or deferred.

Recommendation — R1 — Start with Go

Go offers the best balance of development speed, excellent concurrency model for polling many feeds, mature bot libraries, and easy static binary deployment. Rust is excellent if extreme memory safety / performance is a hard requirement, but Go will get you to a usable MVP faster.

Recommendation — R2 — Telegram First

Ship Telegram only for MVP. It has the best free bot API and the lowest friction for early users. Add Discord once the core is solid. Treat WhatsApp as a later experimental track.

Recommendation — R3 — Shared Feed Model from Day One

Even if you start with per-user polling, design the Feed entity so that multiple subscriptions can point to the same feed. This pays huge dividends when the user base grows.

Recommendation — R4 — Make AI Completely Optional

Core value should work without any AI spend. AI is a powerful differentiator but should never block basic delivery.

Recommendation — R5 — Thin, Documented API for the Dashboard

Expose a clean REST API early. This is the on-ramp for any future JavaScript developers and for a polished web UI.

Recommendation — R6 — Docker + One-Command Self-Host

Aim for `docker compose up` as the primary self-host experience. Provide both SQLite (simple) and Postgres (recommended) paths.

Recommendation — R7 — Invest Early in Observability

Structured logs + a few Prometheus metrics will save enormous time when debugging flaky feeds or delivery issues.

Recommendation — R8 — Keep Special Source Handlers Extensible

YouTube, Reddit, GitHub, etc. will always need custom logic. Make adding a new handler a small, well-defined PR.

17. Appendix

17.1 Glossary

RSS / Atom — Standardized XML formats for syndicating content.
Feed Discovery — Process of finding a feed URL from a normal website URL.
ETag / Last-Modified — HTTP headers used to avoid re-downloading unchanged feeds.
GUID — Globally unique identifier of an item inside a feed (used for deduplication).
Digest — Batched summary of multiple items sent once per day/week instead of real-time.

17.2 Example Commands (Telegram)

`/start` — Welcome + short tutorial
`/add <url>` — Start subscription flow
`/list` — Show active subscriptions
`/unsub <id or name>` — Remove a subscription
`/mute <id>` — Temporarily silence a feed
`/filter <id> only AI and tools` — Set natural-language filter
`/digest on|off` — Toggle daily digest mode
`/status` — Show bot health and your stats

17.3 Suggested Project Layout (Go)

```
cmd/feedbridge/ # main entrypoint
internal/bot/ # Telegram / Discord adapters
internal/feed/ # discovery, parser, poller
internal/ai/ # LLM client + prompts
internal/storage/ # repositories, migrations
internal/worker/ # background jobs
internal/config/ # configuration loading
api/ # optional REST handlers for dashboard
web/ # (future) Next.js dashboard (separate repo or monorepo)
deploy/ # docker-compose, k8s, etc.
docs/ # architecture, self-host guide
```

End of Product Requirements Document — FeedBridge Bot v1.0
This document is living and should be updated as decisions are made and the product evolves.
Questions or suggestions? Discuss with the project owner or open an issue once the repository is public.